

Data Manipulation in Python using Pandas

In Machine Learning, the model requires a dataset to operate, i.e. to train and test. But data doesn't come fully prepared and ready to use. There are discrepancies like Nan/ Null / NA values in many rows and columns. Sometimes the data set also contains some of the rows and columns which are not even required in the operation of our model. In such conditions, it requires proper cleaning and modification of the data set to make it an efficient input for our model. We achieve that by practicing **Data Wrangling** before giving data input to the model.

Today, we will get to know some methods using Pandas which is a famous library of Python. And by using it we can make out data ready to use for training the model and hence getting some useful insights from the results.

Installing Pandas

Before moving forward, ensure that Pandas is installed in your system. If not, you can use the following command to install it:

```
pip install pandas
```

Creating DataFrame

Let's dive into the programming part. Our first aim is to create a Pandas dataframe in Python, as you may know, pandas is one of the most used libraries of Python.

Code:

```
# Importing the pandas library
```

```
import pandas as pd
```

```
# creating a dataframe object
```

```
student_register = pd.DataFrame()
```

```
# assigning values to the
```

```
# rows and columns of the dataframe
```

```
student_register['Name'] = ['Abhijit','Smriti',  
                             'Akash', 'Roshni']
```

```
student_register['Age'] = [20, 19, 20, 14]
```

```
student_register['Student'] = [False, True,  
                               True, False]
```

```
print(student_register)
```

Output:

```
   Name Age Student
0 Abhijit  20   False
1  Smriti  19    True
2  Akash  20    True
3  Roshni  14   False
```

As you can see, the dataframe object has four rows [0, 1, 2, 3] and three columns["Name", "Age", "Student"] respectively. The column which contains the index values i.e. [0, 1, 2, 3] is known as the **index column**, which is a default part in pandas datagram. We can change that as per our requirement too because Python is powerful.

Adding data in DataFrame using Append Function

Next, for some reason we want to add a new student in the datagram, i.e you want to add a new row to your existing data frame, that can be achieved by the following code snippet.

One important concept is that the “dataframe” object of Python, consists of rows which are “series” objects instead, stack together to form a table. Hence adding a new row means creating a new series object and appending it to the dataframe.

Code:

```
# creating a new pandas
# series object

new_person = pd.Series(['Mansi', 19, True],
                        index = ['Name', 'Age',
                                'Student'])

# using the .append() function
# to add that row to the dataframe

student_register.append(new_person, ignore_index = True)

print(student_register)
```

Output:

	Name	Age	Student
0	Abhijit	20	False
1	Smriti	19	True
2	Akash	20	True
3	Roshni	14	False

Data Manipulation on Dataset

Till now, we got the gist of how we can create dataframe, and add data to it. But how will we perform these operations on a big dataset. For this let's take a new dataset

Getting Shape and information of the data

Let's exact information of each column, i.e. what type of value it stores and how many of them are unique. There are three support functions, **.shape**, **.info()** and **.corr()** which output the shape of the table, information on rows and columns, and correlation between numerical columns.

Code:

```
# dimension of the dataframe

print('Shape: ')

print(student_register.shape)

print('-----')

# showing info about the data

print('Info: ')

print(student_register.info())

print('-----')

# correlation between columns

print('Correlation: ')

print(student_register.corr())
```

Output:

```
Shape:
(4, 3)
-----
Info:
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 4 entries, 0 to 3
```

```
Data columns (total 3 columns):
# Column Non-Null Count Dtype
---  ---  ---
0 Name    4 non-null    object
1 Age     4 non-null    int64
2 Student 4 non-null    bool
dtypes: bool(1), int64(1), object(1)
memory usage: 196.0+ bytes
None
```

```
Correlation:
      Age Student
Age    1.000000 0.502519
Student 0.502519 1.000000
```

In the above example, the **.shape** function gives an output (4, 3) as that is the size of the created dataframe.

The description of the output given by .info() method is as follows:

1. **RangeIndex** describes about the index column, i.e. [0, 1, 2, 3] in our datagram. Which is the number of rows in our dataframe.
2. As the name suggests **Data columns** give the total number of columns as output.
3. **Name, Age, Student** are the name of the columns in our data, non-null tells us that in the corresponding column, there is no NA/ Nan/ None value exists. **object, int64** and **bool** are the datatypes each column have.
4. **dtype** gives you an overview of how many data types present in the datagram, which in term simplifies the data cleaning process.
Also, in high-end machine learning models, **memory usage** is an important term, we can't neglect that.

Getting Statistical Analysis of Data

Before processing and wrangling any data you need to get the total overview of it, which includes statistical conclusions like **standard deviation(std), mean and it's quartile distributions**.

```
# for showing the statistical
# info of the dataframe

print('Describe')

print(student_register.describe())
```

Output:

Describe

Age

```
count  4.000000
mean   18.250000
std     2.872281
min    14.000000
25%    17.750000
50%    19.500000
75%    20.000000
max    20.000000
```

The description of the output given by .describe() method is as follows:

1. **count** is the number of rows in the dataframe.
2. **mean** is the mean value of all the entries in the “Age” column.
3. **std** is the standard deviation of the corresponding column.
4. **min** and **max** are the minimum and maximum entry in the column respectively.
5. 25%, 50% and 75% are the **First Quartiles**, **Second Quartile(Median)** and **Third Quartile** respectively, which gives us important info on the distribution of the dataset and makes it simpler to apply an ML model.

Dropping Columns from Data

Let's drop a column from the data. We will use the drop function from the pandas. We will keep axis = 1 for columns.

```
students = student_register.drop('Age', axis=1)
```

```
print(students.head())
```

Output:

```
   Name Student
0  Abhijit  False
1   Smriti   True
2   Akash   True
3  Roshni  False
```

From the output, we can see that the 'Age' column is dropped.

Dropping Rows from Data

Let's try dropping a row from the dataset, for this, we will use drop function. We will keep axis=0.

```
students = students.drop(2, axis=0)
```

```
print(students.head())
```

Output:

	Name	Student
0	Abhijit	False
1	Smriti	True
3	Roshni	False

In the output we can see that the 2 row is dropped.

I